Метрики качества классификации

Доля неправильных ответов

$$\frac{1}{l}\sum_{i=1}^{l}[a(x_i) \neq y_i]$$

Доля правильных ответов

$$accuracy(a, X) = \frac{1}{l}\sum_{i=1}^{l}[a(x_i) = y_i]$$

Проблемы: несбалансированные выборки

Пример 1. Предсказание кликов по рекламе. Если считать положительным примером клик по баннеру, а отрицательным примером показ баннера без клика, то положительных примеров будет гораздо меньше, чем отрицательных.
Пример 2. Медицина диагностика – предсказание редких заболеваний. Так как их мало, мы скорее всего соберём выборку, где много здоровых людей и очень мало примеров людей, которые имеют это редкое заболевание.
Пример 3. Предсказание оттока клиентов. Если оператор будет предсказывать отток клиентов, то, скорее всего, примеров, когда человек ушёл, будет сильно меньше, чем примеров, когда клиент остался клиентом и не стал уходить к конкуренту.

Несбалансированные выборки

Пример:

- Класс -1: 950 объектов
- Класс +1: 50 объектов

$a(x) = -1$
Доля правильных ответов: 0.95
Почему результат нас не устраивает? Модель, которую мы получили, **не несет никакой экономической пользы**, мы не научимся находить людей, которые имеют заболевание, не сможем их лечить вовремя. **Цены ошибок неравнозначны**. Наша задача заключается в поиске больных. Нам страшнее ошибаться - не находить больного, чем здорового человека называть больным (проведём ещё тесты и поймём, что он здоров).
Пусть q_0 – доля объектов самого крупного класса, тогда доля правильных ответов для разумных алгоритмов: $accuracy \in [q_0, 1]$ (а не $[1/2, 1]$).

Разные цены разных типов ошибок.

Нужны дополнительные метрики качества, которые учитывают цены той или иной ошибки.
Пример: кредитный скоринг
Модель 1:
- 80 кредитов вернули
- 20 кредитов не вернули

Модель 2:

- 48 кредитов вернули
- 2 кредита не вернули

Какая модель лучше?
Что хуже:

- Выдать кредит «плохому» клиенту
- Не выдать кредит «хорошему» клиенту

Матрица ошибок

	$y = 1$	$y = -1$
$a(x) = 1$	TP	FP
$a(x) = -1$	FN	TN

Два вида ошибок: ложные срабатывания и ложные пропуски. Для каждого из них нужна своя метрика качества, чтобы измерить, какое количество ошибок какого типа совершается.

Точность и полнота

$$precision(a, X) = \frac{TP}{TP + FP}$$

$$recall(a, X) = \frac{TP}{TP + FN}$$

Точность показывает, какая доля объектов, выделенных классификатором как положительные, действительно является положительными (насколько можно доверять классификатору в случае срабатывания).
Полнота показывает, какая часть положительных объектов была выделена классификатором (на какой доле истинных ответов алгоритм срабатывает).

Примеры

Пример 1. Задача кредитного скоринга. Пусть в задаче кредитного скоринга ставится условие, что неудачных кредитов должно быть не больше 5%. В таком случае задача является задачей максимизации полноты при условии precision $(a, X) \geq 0.95$.
Пример 2. Медицинская диагностика. Необходимо построить модель, которая определяет, есть или нет определенное заболевание у пациента. При этом требуется, чтобы были выявлены как минимум 80% пациентов, которые действительно имеют данное заболевание. Тогда ставят задачу максимизации точности при условии $recall(a, X) \geq 0.8$.

Несбалансированная выборка

	$y = 1$	$y = -1$
$a(x) = 1$	10	20
$a(x) = -1$	90	10000

$accuracy(a, X) = 0.99$
$precision(a, X) = 0.33$
$recall(a, X) = 0.1$
Доля верных ответов равняется 0.99. При этом алгоритм все равно делает 66% ложных срабатываний и выявляет только 10% положительных случаев. Благодаря введению точности и полноты становится понятно, что алгоритм нужно улучшать.
Итак, точность измеряет, насколько можно доверять классификатору, когда он говорит, объект положительный. Полнота показывает, насколько мы полно находим объекты положительного класса. Как объединить в одну?

Арифметическое среднее

$$\frac{1}{2}(precision + recall)$$

Пусть есть алгоритм, у которого $precision = 0.1$, $recall = 1$. Это может быть случай, когда в выборке всего 10% объектов класса +1, а алгоритм является константным и всегда возвращает +1. Очевидно, что этот алгоритм плохой, но введенная выше метрика для него равна $A = 0.55$. В свою очередь другой, гораздо более лучший алгоритм, с $precision = 0.55$ и $recall = 0.55$ также характеризуется $A = 0.55$.

Минимум

$$M = min(precision, recall)$$

В этом случае два алгоритма, для которых точности одинаковы, но отличаются значения полноты, будут иметь одинаковое значение M

1. $precision = 0.4$, $recall = 0.5 = M \Longrightarrow 0.4$,

2. $precision = 0.4$, $recall = 0.9 \Longrightarrow M = 0.4$

Это также недопустимо, так как второй алгоритм существенно лучше первого.

Гармоническое среднее или F-мера

$$F = \frac{2precision * recall}{precision + recall}$$

Для двух упомянутых выше алгоритма значения F-меры, в отличие от M, будут отличаться:

1. $precision = 0.4$, $recall = 0.5 \Longrightarrow F = 0.44$,
2. $precision = 0.4$, $recall = 0.9 \Longrightarrow F = 0.55$

Расширенная F-мера

$$F_\beta = (1 + \beta^2)\frac{precision * recall}{\beta^2 precision + recall}$$

Близка к 0, если хотя бы один из аргументов близок к 0
Изменяя коэффициент можно отдавать предпочтение точности или полноте. Например, при $\beta = 0.5$ важнее оказывается полнота, а в случае $\beta = 2$, наоборот, важнее оказывается точность.

Качество оценок принадлежности классу

$a(x) = [b(x) > t]$
$b(x)$— оценка принадлежности классу + 1
$a(x) = [(w, x) > t]$
$b(x) = (w, x)$ — оценка принадлежности классу +1
Как оценить качество $b(x)$?

Пример – задача кредитного скоринга

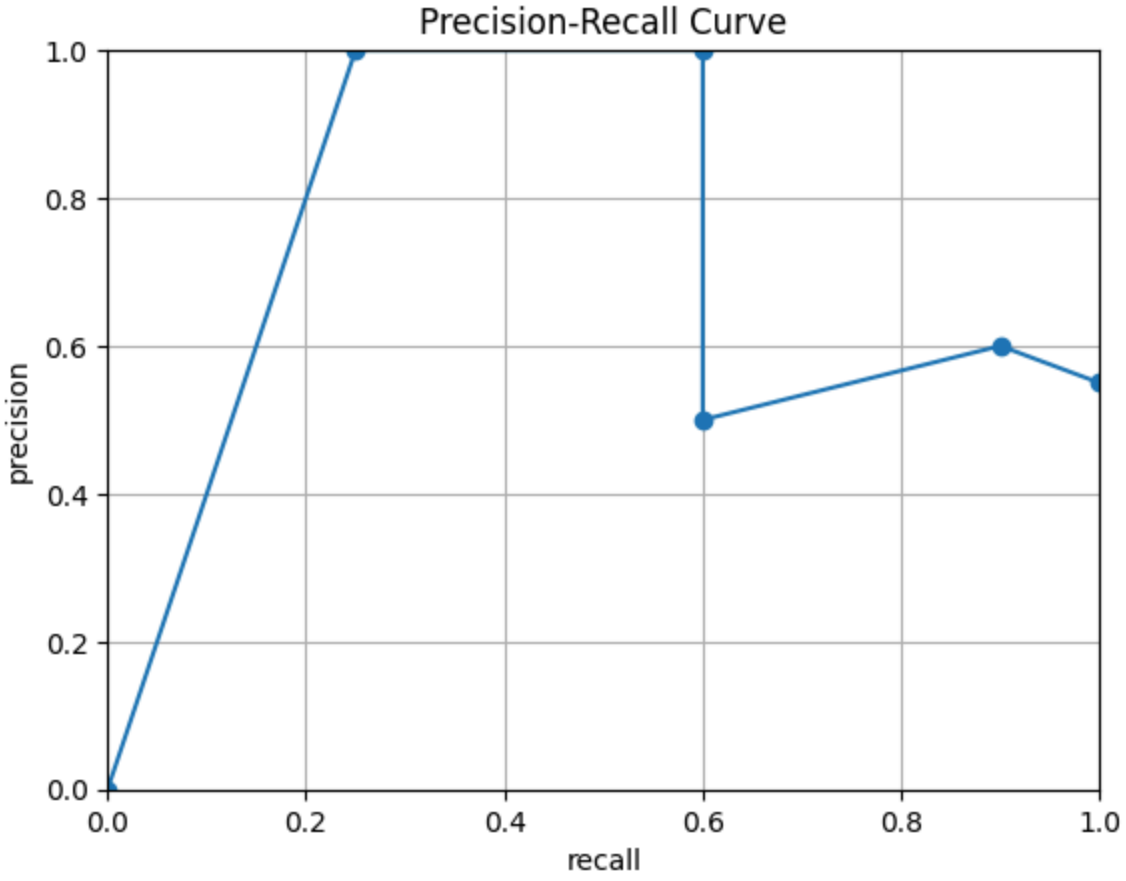
Мы строим модель, но не знаем, какую стратегию выберет банк. Они захотят много рисковать и выдавать много кредитов, или захотят быть консервативными и выдавать мало кредитов? Поэтому мы не можем взять конкретный порог, и при нем измерить точность и полноту. Мы хотим в целом измерить качество $b(x)$ - качество скалярного произведения, если модель линейная. При этом порог реально важен. Действительно, если берем порог высоким, то мы мало объектов отнесем к классу +1, точность будет высокой, потому что мы говорим +1 только там, где уверены что это +1, полнота будет скорее всего низкой. А если берем маленький порог, то мы много объектов отнесем к классу +1, точность будет низкой, но при этом полнота будет высокой.

$b(x)$ — оценка вероятности возврата кредита
$a(x) = [b(x) > 0.5]$
$precision = 0.1, recall = 0.7$
В чём дело — в пороге или в алгоритме?

PR-кривая (кривая точность-полнота)

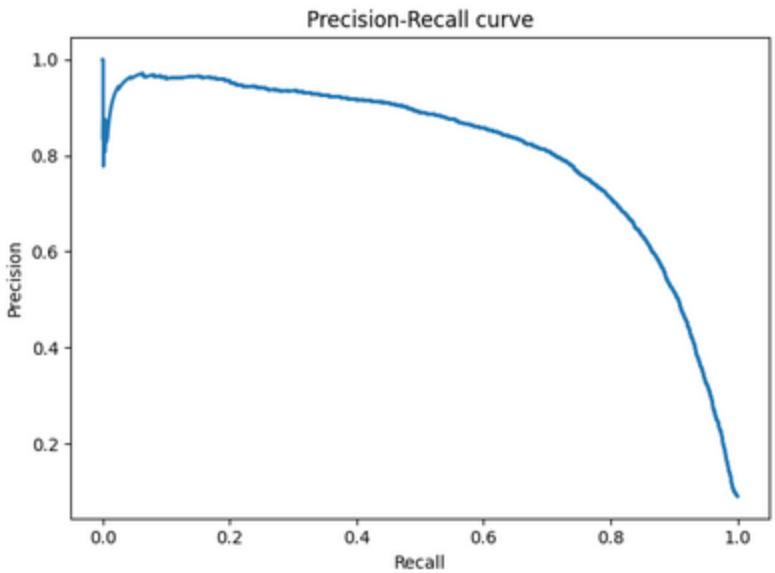
- Ось X - полнота
- Ось Y - точность

При достаточно большом пороге ни один объект не будет отнесен к классу 1. В этом случае и точность и полнота равны 0. При таком пороге, что ровно один объект отнесен к классу 1, точность будет 100% (поскольку этот объект действительно из 1 класса), а полнота - 1/3 (поскольку всего 3 объекта 1 класса, а мы нашли 1). При дальнейшем уменьшении порога уже два объекта отнесены к классу 1, точность также остается 100%, а полнота становится равной 2/3. При таком пороге, что уже три объекта будут отнесены к классу 1, точность становится равной 2/3, а полнота остается такой же. При таком пороге, что четыре объекта отнесены к классу 1, точность уменьшится до 0.5, а полнота опять не изменится. При дальнейшем уменьшении порога уже 5 объектов будут отнесены к 1 классу, полнота станет равной 100%, а точность - 3/5.



$b(x)$	**0.14**	**0.23**	**0.39**	**0.52**	**0.73**	**0.90**
y	0	1	0	0	1	1

В реальных задачах с числом объектов порядка нескольких тысяч или десятков тысяч, кривая точности-полноты выглядит примерно следующим образом. Площадь под этой кривой может быть хорошей мерой качества оценок принадлежности к классу 1.
Левая точка: (0, 0)
Правая точка: (1, г), г — доля положительных объектов
Для идеального классификатора проходит через (1, 1)
AUC-PRC – площадь под PR-кривой



ROC-кривая (receiver operating characteristic)

Ось X - False Positive Rate

$$FPR = \frac{FP}{FP + TN}$$

Ось Y - True Positive Rate

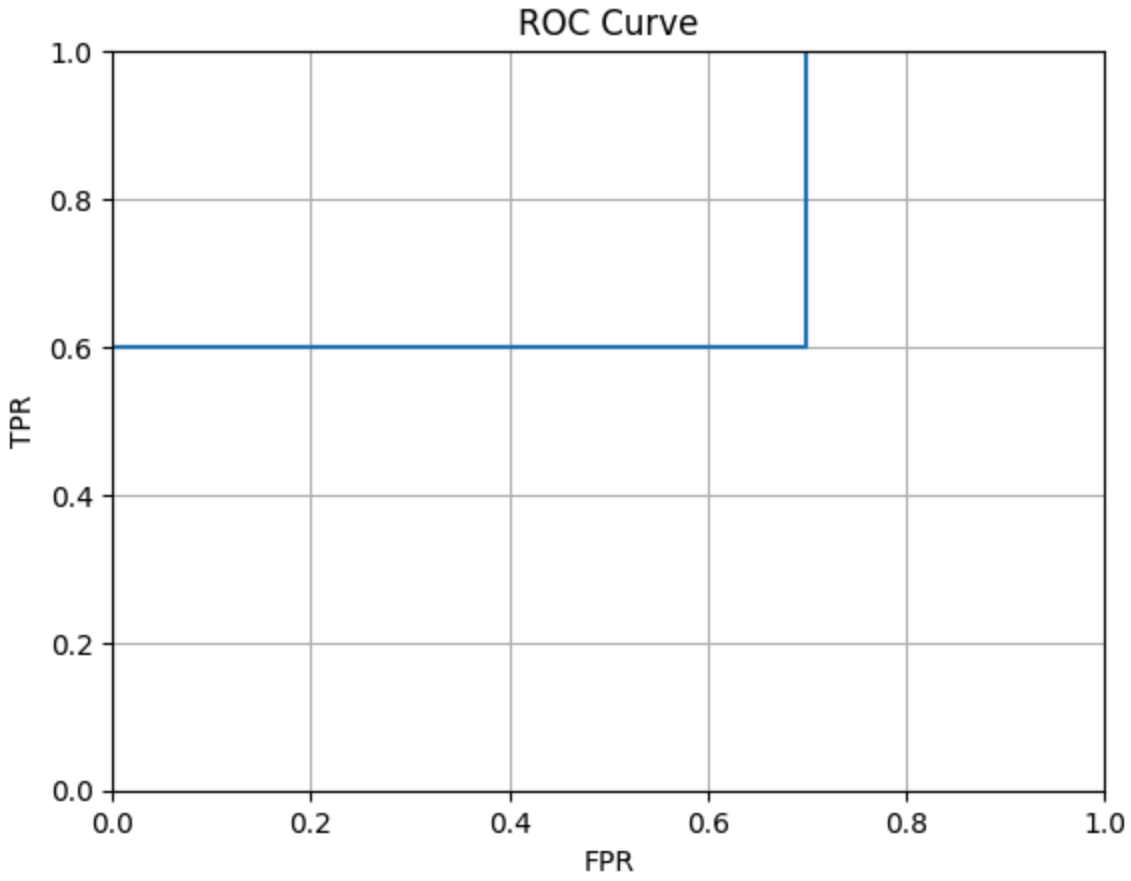
$$TPR = \frac{TP}{TP + FN}$$

1. Если порог большой (например, 1), ни один объект не относится к положительному классу. При нем у нас нет верных срабатываний, нет ложных срабатываний, потому что вообще нет срабатываний. Поэтому и "True Positive Rate" и "False Positive Rate" будут 0, то есть начинаем мы с точки (0,0).
2. Берем порог 0.89. Один объект относится к положительному классу, одно срабатывание, и оно верное. "True Positive Rate" будет равен одной трети, "False

Positive Rate" равен нулю.

3. Берем порог, допустим, 0.7. "True Positive Rate" будет две трети, у нас два верных срабатывания и всего три положительных объекта в выборке. "False Positive Rate" все еще ноль
4. Берем порог 0,5. У нас появляется ложное срабатывание, поэтому "True Positive Rate" будет все еще две трети, "Faise Positive Rate" одна треть.
5. И так далее.

Замечание. Если мы так идем по этой табличке справа налево, встречаем единичку шагаем вверх, встречаем нолик шагаем вправо.
Тогда можно сформулировать алгоритм; проходимся по отсортированной по $b(x)$ выборке и, в зависимости от того, что встречаем один или ноль, двигаемся либо вверх, либо вправо.

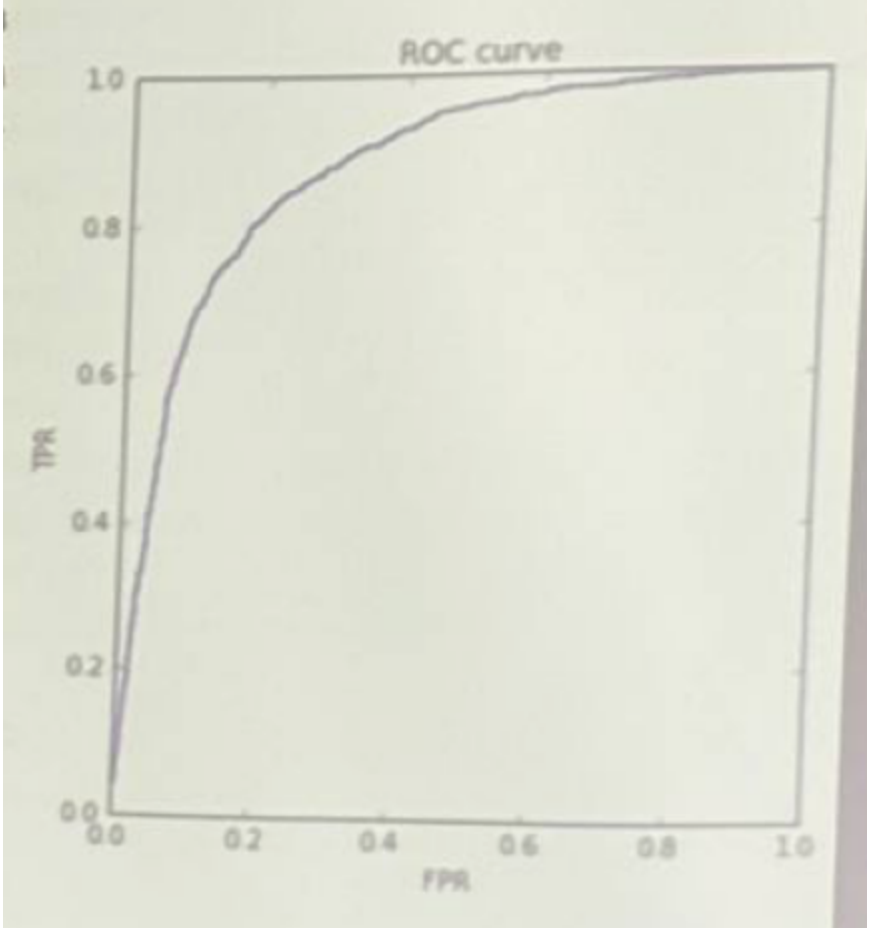


$b(x)$	**0.14**	**0.23**	**0.39**	**0.52**	**0.73**	**0.90**
y	0	1	0	0	1	1

В реальных задачах с десятками тысячами объектов ROC- кривая выглядит следующим образом

1. Всегда начинается в точке (0,0) и заканчивается в точке (1,1)

2. При идеальной классификации у нас будет 100 процентов верных срабатываний и 0 процентов ложных срабатываний, то есть в этом случае получим точку (0,1). Таким образом, у идеального классификатора ROC-кривая будет проходить через точку (0,1) и будет выглядеть как уголок.
3. Что такое плохой классификатор, точнее — худший классификатор? Наверное, это классификатор, который просто перемешал все объекты, где мы проходя по выборке, отсортированной по b(x), встречаем там 0,1, 0,1,0,1 и так далее. Тогда мы будем идти «лесенкой» вдоль диагонали. Площадь под ней равна примерно одной второй.
4. Итак, для идеальной модели площадь под ROC-кривой это 1, для худшей модели площадь под ROC-кривой где-то 0.5.



Итог по ROC и PR кривой

Для ROC-кривой

$$FPR = \frac{FP}{FP + TN}$$

$$TPR = \frac{TP}{TP + FN}$$

AUC-ROC не поменяется при изменении баланса классов
Идеальный алгоритм: AUC-ROC = 1

Худший алгоритм: AUC-ROC = 2

Для PR-кривой

$$precision(a, X) = \frac{TP}{TP + FP}$$

$$recall(a, X) = \frac{TP}{TP + FN}$$

Точность поменяется при изменении баланса классов
AUC - PRC зависит от баланса классов
Проще интерпретировать, если выборка несбалансированная

Пример

Возьмем порог, при котором мы относим к положительному классу 50095 объектов, из которых 95 верных срабатываний и 50000 ложных срабатываний.
Точность очень низкая, потому что среди всех срабатываний очень мало верных срабатываний. Полнота очень высокая и это логично: мы нашли почти весь положительный класс. "True Positive Rate" большой - 95%. ("True Positive Rate" и полнота - это одно и то же). "False Positive Rate" очень низкий - 5%. Площадь под ROC-кривой довольно большая.
AUC-ROC = 0.95
AUC-PRC = 0.001
Выберем конкретный классификатор
$a(x) = 1\$ - 50095 объектов Из них: FP = 50000, TP = 95 TPR = 0.95 FPR = 0.05$precision$ = 0.0019
$recall$ = 0.95
$b(x) = 1$

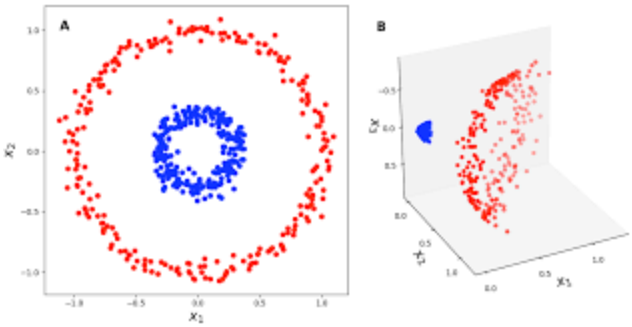
Метрика, которая называется площадь под ROC-кривой очень популярна, когда мы измеряем качество моделей и хотим посмотреть на качество безотносительно какого-то конкретного порога. Она не зависит от баланса классов и хорошо интерпретируется. Проблемы могут возникнуть при очень несбалансированных выборах, следовательно, в этом случае стоит быть аккуратным при интерпретации значений площади под ROC-кривой и площади под "Precision Recall" кривой. "Precision Recall" кривая выразительнее в этом случае.

Основная проблема линейного классификатора – неразделимость выборки: объекты разных классов сильно перемешаны и они не разделяются прямой или плоскостью.
В этом случае линейный классификатор будет давать большое число ошибок.
В этом случае можно применить «трюк с ядром» (Kernel Trick). Kernel Trick заключается в преобразовании пространства объектов. Объект описывается точкой в пространстве.

Трансформируем пространство: поворачиваем, сжимаем, растягиваем. После этого классы становятся разделимыми с помощью плоскости.

Kernel Trick

Представьте, что эти точки нарисованы на плёнке, которую можно растягивать. Вставайте ногами в центр и тяните все четыре угла вверх (есть математические формул, которые осуществляют такое преобразование). Все зеленые точки останутся на полу, а все синие точки переместятся выше. Тогда можно будет построить плоскость, разделяющую синие и зеленые точки (объекты).



Логистическая регрессия

По результатам входного тестирования студента (оценка по 10-балльной шкале, может быть дробной - 4.5, 4.7, например) предсказать, выпустится ли он из университета (да 1 или нет 0).
В реальности правильнее для каждого возможного значения оценки выдать некоторую вероятность.

Кредитный скоринг

Допустим, мы построим некоторую модель $b(x)$, которая выдает вероятность того, что клиент вернет кредит.
Тогда можно было бы использовать эту модель, чтобы принимать решение о выдаче следующим образом: пусть финансисты определили, что 10% кредитов, которые не вернут – это нормально, то есть остальные 90% кредитов компенсируют расходы на эти 10%.
Тогда мы можем сказать, что выдаем кредит только тем клиентам, у которых вероятность возврата 90 процентов или больше ($b(x) \geq 0.9$).

Будем говорить, что модель $b(x)$ предсказывает вероятность того, что объект относится к положительному классу, если выполняется следующее. Рассмотрим модель $b(x)$ и некоторую вероятность p. Возьмем все объекты, у которых $b(x) = p$. Если среди этих объектов оказывается реально $p\%$ положительных, то будем считать, что модель корректно предсказывает вероятности.

Для примера с кредитным скорингом: если мы выдаем кредиты тем, у кого вероятность возврата 90% или больше, мы ожидаем, что невозвратов будет не больше 10 процентов.

Применим к скалярному произведению какую-нибудь функцию, которая переведет его на отрезок $[0, 1]$.
Таких функций много.
Одна из популярных – сигмоида.

$$\sigma(< w, x >) = \frac{1}{1 + \exp(- < w, x >)}$$

На вход принимает любое вещественные число, а выдает числа от нуля до единицы.
Для очень маленьких чисел она выдает ноль, для очень больших выдает единицу, около нуля происходит переход.
Если мы применим сигмоиду к нашему скалярному произведению в нашей задаче и правильно подберем параметры w_1 и w_0, то получим функцию, которая в общем соответствует тому, что мы хотели.

Нужно научиться подбирать параметры w_1 и w_0, чтобы получить именно такое расположение функции.
Итак, модель для оценивания вероятностей $b(x) = \sigma(< w, x >)$

Как обучать?
Мы хотим предъявить следующие требования:

- если $y_i = +1$, то $\sigma(< w, x >) \to 1$ или $< w, x_i > \to +\infty$
- если $y_i = -1$, то $\sigma(< w, x >) \to 0 или\$ < w, x_i > \to -\infty$

Таким образом, нужно сделать отступы на всех объектах максимальными

$$y_i < w, x_i > \to \max_w$$

Запишем подходящий функционал, например, след вида:

$$-\sum_{i=1}^{l}([y_i = 1]\sigma(< w, x_i >) + [y_i = -1](1 - \sigma(< w, x_i >))) \to \min_w$$

(в квадратных скобках указаны «индикаторы», к какому классу принадлежит объект)
Для положительных объектов вероятность должна как можно больше - максимизируем сигмоиду, для отрицательных – как можно меньше – минимизируем сигмоиду.
Предположим теперь, что случилась какая-то очень серьезная ошибка — есть положительный объект, на нём y_i равняется +1, а модель абсолютно уверена, что это отрицательный объект - она выдаёт для этого объекта сигмоиду, равную нулю. Почему это очень плохо? Мы ввели вероятности, для того чтобы измерять некоторую неуверенность модели, а здесь получается, что модель абсолютно уверена в неправильном решении.
Как наш функционал штрафует в этом случае модель?
Если $y_i = +1$ и $\sigma(< w, x_i >) = 0$, то штраф равен 1 (в идеальном случае – 1, самое меньшее, а у нас 0). Это слишком слабый штраф.

Введем функцию потерь log-loss (если модель крайне уверена в неправильном (положительном) ответе, то штраф равен бесконечности)
$L(y, z) = -[y = 1]\log z - [y = -1]\log(1 - z)$
Если $y_i = +1$ и $\sigma(< w, x >) = 0$, то штраф равен $-\log 0 = +\infty$

$$-\sum_{i=1}^{l}([y_i = 1]\log(\sigma(< w, x_i >)) + [y_i = -1]\log(1 - \sigma(< w, x_i >))) \to \min_w$$

$$-\sum_{i=1}^{l}([y_i = 1]\log(\sigma(< w, x_i >)) + [y_i = -1]\log(1 - \sigma(< w, x_i >))) =$$

$$= -\sum_{i=1}^{l}([y_i = 1]\log(\frac{1}{1 + \exp(- < w, x_i >)}) + [y_i = -1]\log(1 - \frac{1}{1 + \exp(- < w, x_i >)})) =$$

Приведем то, что стоит под $\log$во втором слагаемом к общему знаменателю и поделим на exp$(- < w, x_i >)$

$$-\sum_{i=1}^{l}([y_i=1]\log(\frac{1}{1+\exp(-<w,x_i>)})+[y_i=-1]\log(\frac{1}{1+\exp(<w,x_i>)}))=$$

«Внесем» минус под логарифмы и «перевернем» дроби

$$-\sum_{i=1}^{l}([y_i=1]\log(1+\exp(-<w,x_i>))+[y_i=-1]\log(1+\exp(<w,x_i>)))=$$

$$=-\sum_{i=1}^{l}\log(1+\exp(-y_i<w,x_i>))$$

Метод опорных векторов

Решаем задачу бинарной классификации
$\mathbf{Y}=-1,+1$
Минимизация верхней оценки

$$\frac{1}{l}\sum_{i=1}^{l}\max(0,1-y_i<w,x_i>)\to min_w$$

Будем максимизировать отступ классификатора – расстояние от гиперплоскости до ближайшего объекта (иногда это называют зазором классификатора).
При этом будем стараться сделать поменьше ошибок.

Простой случай

Будем считать, что выборка линейно разделима (существует линейный классификатор, который не делает ошибок)

Линейно разделимый случай

Требование 1. На всех объектах – правильный ответ, то есть отступ положительный)
$y_i(<w,x_i>+w_0)>0$ для всех $i=1,2,\ldots,l$
Требование 2. Максимальный отступ классификатора

$$\frac{1}{||w||}\to max_w$$

При условии, что

$$\min_{i=1,\ldots,l}|<w,x_i>+w_0|=1$$

Отступ классификатора

Расстояние от точки до гиперплоскости $<w,x_i>+w_0=0$:

$$\frac{|<w, x_i>| + w_0}{||w||}$$

Отступ классификатора:

$$\min_{i=1,\ldots,l} \frac{|<w, x_i>| + w_0}{||w||}$$

Слишком сложно для дальнейшей работы, будем упрощать.
Линейный классификатор имеет вид:

$$a(x) = sign(<w, x_i> + w_0)$$

Если мы поделим w и w_0 на число $a > 0$, то выходы классификатора никак не поменяются.

$$a(x) = sign(\frac{<w, x_i> + w_0}{a}) = sign(<w, x_i> + w_0)$$

Поделим w и w_0 на

$$\min_{i=1,\ldots,l} |<w, x_i> + w_0| > 0$$

Замечание. Мы договорились, что можно провести прямую так, что она не проходит ни через один объект и при этом не допускает ошибок. Поэтому можно предполагать, что этот минимум будет больше нуля для множества корректных классификаторов.
Тогда

$$\min_{i=1,\ldots,l} |<w, x_i> + w_0| = 1$$

Действительно, мы взяли минимальное значение выхода модели и поделили все выходы на это минимальное значение. Поэтому минимальный выход после этого станет равен единице.
Отступ классификатора:

$$\min_{i=1,\ldots,l} \frac{|<w, x_i> + w_0|}{||w||} = \frac{\min_{i=1,\ldots,l} |<w, x_i> + w_0|}{||w||} = \frac{1}{||w||}$$

Последнее требование можно упростить:
$<w, x_i> + w_0 >= 1$
Если совместить его с тем, что мы минимизируем $||w||$

$$\begin{cases} ||w||^2 \to min_{w,w_0} \\ y_i(<w, x_i> + w_0) >= 1 \end{cases}$$

Это задача метода опорных векторов (SVM) для линейно разделимого случая.

Линейно неразделимый случай

Д/з

1. Прочитайте материалы по использованию метода KNN для классификации изображений. Как в этих случаях определяется расстояние? (на хорошую оценку)
2. Для объектов из таблицы с помощью некоторого алгоритма были получены вероятности принадлежности классу 1. Истинные метки классов объектов также известны. Найдите площадь под ROC-кривой.

Объекты	Вероятность принадлежности классу 1	Истинные метки классов
A	0.6	1
B	0.81	1
C	0.5	0
D	0.9	1
E	0.7	0
F	0.75	0